3TIB_03_ChronicKidneyDisease

sorry saya lupa menambahkan file excelnya , berikut adalah file excel data saya tolong ubah pada bagian kolom yang awalnya tidak terdefinisi di menjadi terdefinisi pada codingan berikut

Tampilkan kode

Berikut adalah daftar kolom yang tersedia pada file Excel Anda (`3TIB_03_ChronicKidneyDisease.xlsx`):

- `id` , `age` , `bp` , `sg` , `al` , `su` , `rbc` , `pc` , `pcc` , `ba` , `bgr` , `bu` , `sc` , `sod` , `pot` , `hemo` , `pcv` , `wc` , `rc` , `htn` , `dm` , `cad` , `appet` , `pe` , `ane` , `classification`

Sebagai contoh, jika Anda ingin membuat histogram untuk kolom umur (`age`), Anda bisa menggunakan kode berikut yang sudah disesuaikan:

Sebagai contoh, jika Anda ingin membuat histogram untuk kolom umur (`age`), Anda bisa menggunakan kode berikut yang sudah disesuaikan:

Python

```python
import matplotlib.pyplot as plt
import pandas as pd

# 1. Membaca file Excel
df = pd.read_excel('3TIB_03_ChronicKidneyDisease.xlsx')

# Tampilkan beberapa baris pertama untuk verifikasi data
print(df.head())

# 2. Membuat Histogram untuk kolom 'age' (Umur)
plt.figure(figsize=(8, 5))

plt.hist(df['age'], bins=10, color='skyblue', edgecolor='black')

# Menambahkan judul dan label dengan rumus / informasi yang jelas
plt.title('Histogram Distribusi Usia (Age)', fontsize=14)
plt.xlabel('Usia ($x$)', fontsize=12)
plt.ylabel('Frekuensi ($f_i$)', fontsize=12)

# Menambahkan grid agar mudah dibaca
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Menampilkan grafik
plt.show()
```

Penjelasan Rumus Statistik untuk Histogram Ini:

- **Lebar Bin (w):** Dihitung berdasarkan rentang nilai usia minimum ($x_{\min}$) hingga maksimum ($x_{\max}$) dibagi jumlah kelas ($k = 10$):

$$w = \frac{x_{\max} - x_{\min}}{10}$$

- **Frekuensi (f_i):** Jumlah pasien yang masuk ke dalam setiap kelompok rentang usia tertentu:

$$f_i = \sum_{j=1}^{N} I(x_j \in \text{Bin}_i)$$

Anda dapat mengganti `'age'` dengan kolom numerik lain yang ada di dalam dataset Anda (misalnya `'bp'` untuk tekanan darah, `'bgr'` untuk kadar gula darah, dll.).



ok saya salah dalam beberapa hal , yaitu pada bagian foto ini , dimana saya cuman mau menghitung 250 data saja jadi tampilkan sesuai dengan yang ingin ditampilkan di dalam foto

Tampilkan kode

Berdasarkan dokumen yang Anda lampirkan pada gambar, analisis statistika deskriptif difokuskan pada **250 pasien** pertama dengan variabel utama yaitu `age` (usia), `bp` (tekanan darah), dan `sc` (kreatinin serum).

Berikut adalah kode Python lengkap untuk membatasi data sebanyak 250 baris, menghitung seluruh ukuran statistik deskriptif yang diminta (jumlah, min, max, range, Q1, Q3, mean, median, modus, varians, dan standar deviasi menggunakan **Persamaan 3-1 hingga 3-3**), serta menampilkan visualisasi **Histogram** untuk ketiga variabel tersebut.

Kode Python Lengkap (Sesuai Persyaratan Dokumen)

Python

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# 1. Membaca file Excel dan mengambil 250 data pertama
df = pd.read_excel('3TIB_03_ChronicKidneyDisease.xlsx')
df_250 = df.head(250)

# Kolom yang dianalisis sesuai dokumen
kolom_target = ['age', 'bp', 'sc']

# 2. Menghitung Statistika Deskriptif untuk 250 Pasien
print('=' * 60)
print('TABEL RINGKASAN STATISTIKA DESKRIPTIF (250 PASIEN)')
print('=' * 60)

stats_summary = []

for col in kolom_target:
  data = df_250[col].dropna()  # Menghapus nilai kosong jika ada
  n = len(data)
  x_mean = data.mean()  # Persamaan 3-1: Mean (x_bar)
  variance = data.var(ddof=1)  # Persamaan 3-2: Varians sampel (s^2)
  std_dev = data.std(ddof=1)  # Persamaan 3-3: Standar deviasi sampel (s)

  minimum = data.min()
  maximum = data.max()
  data_range = maximum - minimum
  q1 = data.quantile(0.25)
  median = data.quantile(0.50)  # Q2
  q3 = data.quantile(0.75)
  mode = data.mode()[0] if not data.mode().empty else np.nan
```

```
    stats_summary.append({
        'Variabel': col,
        'Jumlah (n)': n,
        'Mean (x̄)': round(x_mean, 2),
        'Median': round(median, 2),
        'Modus': round(mode, 2),
        'Min': minimum,
        'Max': maximum,
        'Range': data_range,
        'Q1': q1,
        'Q3': q3,
        'Varians (s²)': round(variance, 2),
        'Std Dev (s)': round(std_dev, 2),
    })

df_stats = pd.DataFrame(stats_summary)
print(df_stats.to_string(index=False))
print('=' * 60)

# 3. Membuat Visualisasi Histogram untuk ke-3 Variabel
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
titles = [
    'Histogram Usia (age)',
    'Histogram Tekanan Darah (bp)',
    'Histogram Kreatinin Serum (sc)',
]
x_labels = ['Usia (tahun)', 'Tekanan Darah (mmHg)', 'Kreatinin Serum (mg/dL)']

for i, col in enumerate(kolom_target):
    axes[i].hist(
        df_250[col].dropna(), bins=10, color='skyblue', edgecolor='black'
    )
    axes[i].set_title(titles[i], fontsize=12, fontweight='bold')
    axes[i].set_xlabel(x_labels[i], fontsize=10)
    axes[i].set_ylabel('Frekuensi ($f_i$)', fontsize=10)
    axes[i].grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

Penjelasan Persamaan Matematis yang Digunakan:

1. **Mean (Rata-rata) — Persamaan 3-1:**

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

Digunakan untuk mencari nilai rata-rata dari data pasien ke-i sampai ke-n ($n = 250$).

2. **Varians Sampel — Persamaan 3-2:**

$$s^2 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n-1}$$

Mengukur tingkat penyebaran atau keragaman data terhadap nilai rata-ratanya dengan derajat kebebasan $n - 1$.

3. **Standar Deviasi Sampel — Persamaan 3-3:**

$$s = \sqrt{\frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n-1}}$$

Akar kuadrat dari varians sampel untuk mengembalikan satuan ukuran kembali ke satuan awal data asli.

buatkan saya juga boxflot yang diminta pada gambar sebelumnya

Tampilkan kode <>

Berikut adalah kode lengkap untuk membuat **Boxplot** sesuai dengan permintaan pada gambar dokumen Anda sebelumnya (untuk **250 pasien** pertama pada variabel `age`, `bp`, dan `sc`).

Boxplot ini berguna untuk melihat distribusi data sekaligus mendeteksi adanya nilai ekstrem (*outlier*) pada masing-masing variabel.

Kode Python untuk Membuat Boxplot

Python

```python
import matplotlib.pyplot as plt
import pandas as pd

# 1. Membaca file Excel dan mengambil 250 data pertama
df = pd.read_excel('3TIB_03_ChronicKidneyDisease.xlsx')
df_250 = df.head(250)

# Kolom target sesuai dengan dokumen (age, bp, sc)
kolom_target = ['age', 'bp', 'sc']
titles = [
    'Boxplot Usia (age)',
    'Boxplot Tekanan Darah (bp)',
    'Boxplot Kreatinin Serum (sc)',
]
y_labels = ['Usia (tahun)', 'Tekanan Darah (mmHg)', 'Kreatinin Serum (mg/dL)']

# 2. Membuat Visualisasi Boxplot (1 Baris, 3 Kolom)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

```
for i, col in enumerate(kolom_target):
  # Membuat boxplot dengan orientasi vertikal
  bp = axes[i].boxplot(
      df_250[col].dropna(),
      patch_artist=True,
      boxprops=dict(facecolor='lightblue', color='black'),
      medianprops=dict(color='red', linewidth=1.5),
      flierprops=dict(
          marker='o', color='black', alpha=0.5, markersize=5
      ),  # Untuk outlier
  )

  axes[i].set_title(titles[i], fontsize=12, fontweight='bold')
  axes[i].set_ylabel(y_labels[i], fontsize=10)
  axes[i].set_xticklabels([col])  # Label sumbu x
  axes[i].grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

Penjelasan Komponen Utama dalam Boxplot:

1. **Garis Merah di dalam Kotak:** Menunjukkan nilai **Median** (Q_2 atau persentil ke-50).
2. **Kotak (Box):** Menunjukkan rentang interkuartil ($IQR = Q_3 - Q_1$), di mana bagian bawah kotak adalah **Kuartil Pertama (Q_1)** dan bagian atas kotak adalah **Kuartil Ketiga (Q_3)**.
3. **Whiskers (Garis Vertikal di luar Kotak):** Menunjukkan batas sebaran data normal di luar Q_1 dan Q_3.
4. **Titik-titik di Luar Whisker (jika ada):** Menunjukkan **nilai ekstrem (*outlier*)**, yaitu data yang nilainya sangat jauh dari kelompok data lainnya (sering ditemukan pada variabel tekanan darah atau kreatinin serum).



apakah ini perlu dibuat visualisasinya?

Berdasarkan dokumen yang Anda lampirkan (pada bagian yang disorot kuning), jawabannya adalah **ya, sangat disarankan dan bahkan biasanya diperlukan sebagai pendukung visual.**

Dokumen tersebut menyatakan bahwa:

> `"Histogram dan Q-Q plot digunakan sebagai pendukung secara visual."`

Meskipun pengujian normalitas menggunakan **Uji Shapiro-Wilk** menghasilkan angka statistik (W dan *p-value*) secara numerik/matematis, visualisasi berupa **Histogram** dan **Q-Q Plot** mutlak diperlukan di dalam bab analisis data untuk memperkuat argumen apakah data benar-benar berdistribusi normal atau tidak secara kasat mata.

Ingin dibuatkan syntax Python-nya?

Jika Anda ingin membuat uji normalitas Shapiro-Wilk beserta visualisasi **Histogram** dan **Q-Q Plot** untuk 250 data pasien pada variabel `age`, `bp`, dan `sc`, beri tahu saya agar saya buatkan kodenya beserta penjelasan rumusnya!

buatkan saya visualisasinya dalam bentuk phyton sesuai dengan ketentuan yang diminta pada uji normalitas

Tampilkan kode <>

Berikut adalah kode Python lengkap untuk membuat visualisasi pendukung uji normalitas sesuai dengan ketentuan dokumen Anda (yaitu **Histogram** dengan kurva distribusi normal dan **Q-Q Plot**) untuk 250 data pasien pertama pada variabel `age`, `bp`, dan `sc`.

Di dalam kode ini juga disertakan perhitungan **Uji Shapiro-Wilk (Statistik W)** secara otomatis menggunakan library `scipy.stats` beserta perbandingan nilai p-value terhadap taraf signifikansi $\alpha = 0,05$.

Kode Python Visualisasi dan Uji Normalitas

Python

```python
import matplotlib.pyplot as plt
import scipy.stats as stats
import pandas as pd

# 1. Membaca file Excel dan mengambil 250 data pertama
df = pd.read_excel('3TIB_03_ChronicKidneyDisease.xlsx')
df_250 = df.head(250)

# Variabel target sesuai dokumen
kolom_target = ['age', 'bp', 'sc']
labels = ['Usia (age)', 'Tekanan Darah (bp)', 'Kreatinin Serum (sc)']

# 2. Loop untuk melakukan Uji Shapiro-Wilk dan Membuat Visualisasi (Histogram & Q-Q
for i, col in enumerate(kolom_target):
  data = df_250[col].dropna()
```

```
# Uji Shapiro-Wilk (Persamaan 3-4)
w_stat, p_value = stats.shapiro(data)

print(f'--- Hasil Uji Normalitas Shapiro-Wilk: {labels[i]} ---')
print(f'Statistik Uji W : {w_stat:.4f}')
print(f'p-value         : {p_value:.4e}')
if p_value >= 0.05:
  print(
      'Keputusan       : H0 diterima (Data berdistribusi normal, p-value >='
      ' 0.05)\n'
  )
else:
  print(
      'Keputusan       : H0 ditolak (Data tidak berdistribusi normal,'
      ' p-value < 0.05)\n'
  )

# Membuat Visualisasi (1 baris untuk Histogram, 1 baris untuk Q-Q Plot)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# A. Histogram dengan Kurva Normal (Pendukung Visual)
axes[0].hist(
    data,
    bins=10,
    density=True,
    color='skyblue',
    edgecolor='black',
    alpha=0.7,
    label='Histogram Data',
)
# Menambahkan garis kurva distribusi normal teoretis
xmin, xmax = axes[0].get_xlim()
x = pd.Series(range(int(xmin), int(xmax) + 1))
p = stats.norm.pdf(x, data.mean(), data.std())
axes[0].plot(x, p, 'r-', linewidth=2, label='Kurva Normal Teoretis')

axes[0].set_title(
    f'Histogram & Kurva Normal: {labels[i]}', fontsize=12, fontweight='bold'
)
axes[0].set_xlabel(labels[i], fontsize=10)
axes[0].set_ylabel('Densitas Probabilitas', fontsize=10)
axes[0].legend()
axes[0].grid(axis='y', linestyle='--', alpha=0.7)
```

```
    axes[0].grid(axis='y', linestyle='--', alpha=0.7)

    # B. Q-Q Plot (Pendukung Visual)
    stats.probplot(data, dist='norm', plot=axes[1])
    axes[1].set_title(f'Q-Q Plot: {labels[i]}', fontsize=12, fontweight='bold')
    axes[1].set_xlabel('Kuantil Teoretis', fontsize=10)
    axes[1].set_ylabel('Kuantil Sampel Data', fontsize=10)
    axes[1].grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.show()
```

Penjelasan Bentuk Visualisasi:

1. **Histogram + Kurva Normal:** Batang histogram dibandingkan dengan garis kurva berbentuk lonceng (*bell-shaped curve*). Jika batang-batang histogram simetris mengikuti garis kurva merah, maka secara visual data mendekati normal.
2. **Q-Q Plot:** Grafik penyebaran titik data terhadap garis diagonal merah lurus. Jika titik-titik data berhimpit atau mengikuti garis diagonal tersebut, maka secara visual data dapat dikatakan berdistribusi normal.



apakah ini perlu di visualisasikan?

Berdasarkan kalimat terakhir pada dokumen yang Anda lampirkan (pada bagian yang disorot biru):

> "Hubungan antarvariabel divisualisasikan menggunakan *scatter plot*."

Jawabannya adalah **ya, perlu divisualisasikan.**

Untuk bagian **Analisis Korelasi** ini, visualisasi yang tepat adalah **Scatter Plot (Diagram Pencar)** untuk melihat pola hubungan, arah (positif/negatif), serta kekuatan hubungan antar pasangan variabel (`age-bp` , `age-sc` , dan `bp-sc`).

Apakah Anda ingin dibuatkan kode Python untuk menghitung korelasi (Pearson/Spearman sesuai hasil uji normalitas sebelumnya) sekaligus membuat visualisasi *scatter plot* untuk pasangan variabel tersebut?

buatkan saya syntax phytonya sesuai dengan ketentuan dari ketentuan pada analisis korelasi

Tampilkan kode

Berikut adalah kode Python lengkap untuk melakukan **Analisis Korelasi** sesuai dengan ketentuan pada dokumen Anda (untuk 250 data pasien pertama pada pasangan variabel `age-bp`, `age-sc`, dan `bp-sc`).

Kode ini secara otomatis mendeteksi normalitas data: jika kedua variabel berdistribusi normal, digunakan **Korelasi Pearson (Persamaan 3-5)**; jika tidak, digunakan **Korelasi Spearman (Persamaan 3-6)**, dilengkapi dengan visualisasi **Scatter Plot**.

Kode Python Analisis Korelasi & Scatter Plot

Python

```python
import matplotlib.pyplot as plt
import scipy.stats as stats
import pandas as pd
import seaborn as sns

# 1. Membaca file Excel dan mengambil 250 data pertama
df = pd.read_excel('3TIB_03_ChronicKidneyDisease.xlsx')
df_250 = df.head(250)

# Pasangan variabel yang akan dianalisis korelasi dan dibuat scatter plot-nya
pasangan_variabel = [('age', 'bp'), ('age', 'sc'), ('bp', 'sc')]

labels = {
    'age': 'Usia (tahun)',
    'bp': 'Tekanan Darah (mmHg)',
    'sc': 'Kreatinin Serum (mg/dL)',
}

# 2. Perhitungan Korelasi & Visualisasi Scatter Plot
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i, (x_col, y_col) in enumerate(pasangan_variabel):
  data = df_250[[x_col, y_col]].dropna()
  x = data[x_col]
  y = data[y_col]

  # Cek normalitas untuk menentukan penggunaan Pearson (Persamaan 3-5) atau Spearman
  _, p_x = stats.shapiro(x)
  _, p_y = stats.shapiro(y)
```

```
    if p_x >= 0.05 and p_y >= 0.05:
      # Persamaan 3-5: Korelasi Pearson (r)
      corr, p_val = stats.pearsonr(x, y)
      metode = 'Pearson (r)'
    else:
      # Persamaan 3-6: Korelasi Spearman (rho)
      corr, p_val = stats.spearmanr(x, y)
      metode = 'Spearman (ρ)'

    print(f'--- Analisis Korelasi: {x_col} vs {y_col} ---')
    print(f'Metode yang Digunakan : {metode}')
    print(f'Koefisien Korelasi    : {corr:.4f}')
    print(f'p-value               : {p_val:.4e}\n')

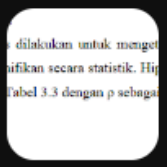
    # Membuat Scatter Plot (Visualisasi Pendukung)
    sns.scatterplot(
        x=x, y=y, ax=axes[i], color='teal', alpha=0.7, edgecolor='k'
    )
    sns.regplot(
        x=x,
        y=y,
        ax=axes[i],
        scatter=False,
        color='red',
        line_kws={'linestyle': '--'},
    )

    axes[i].set_title(
        f'{x_col.upper()} vs {y_col.upper()}\n({metode} = {corr:.2f})',
        fontsize=11,
        fontweight='bold',
    )
    axes[i].set_xlabel(labels[x_col], fontsize=10)
    axes[i].set_ylabel(labels[y_col], fontsize=10)
    axes[i].grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

Penjelasan Logika & Rumus yang Diterapkan:

1. **Seleksi Metode Otomatis:** Program mengecek apakah kedua variabel berdistribusi normal (p-value $\geq 0,05$ pada uji Shapiro-Wilk). Jika ya, menggunakan rumus **Pearson (r)**; jika salah satu/keduanya tidak normal, otomatis beralih menggunakan rumus **Spearman (ρ)**.
2. **Scatter Plot + Garis Tren:** Grafik sebar (*scatter plot*) menampilkan titik-titik data antarvariabel disertai garis putus-putus merah (garis regresi/tren) untuk memperlihatkan apakah arah hubungan antarvariabel bernilai positif (cenderung naik) atau negatif (cenderung turun).


dilakukan untuk menget
ifikan secara statistik. Hip
Tabel 3.3 dengan ρ sebaga



apakah bagian ini perlu di isualissasikan?

Berdasarkan dokumen yang Anda lampirkan pada bagian **Pengujian Hipotesis**, jawabannya adalah **tidak memerlukan bentuk visualisasi grafik** (seperti histogram atau *scatter plot*).

Bagian ini difokuskan pada pengujian keputusan statistik secara numerik, di mana hasilnya disajikan dalam bentuk **Tabel 3.3 (Rumusan Hipotesis)** serta nilai statistik uji t (Persamaan 3-7) dan perbandingan nilai p-value terhadap taraf signifikansi $\alpha = 0,05$.

Ingin dibuatkan syntax Python untuk Pengujian Hipotesis ini?

Jika Anda ingin melihat hasil perhitungan statistik uji t beserta keputusan uji hipotesis (H_0 diterima atau ditolak) untuk 250 pasien pertama pada pasangan variabel `age–bp`, `age–sc`, dan `bp–sc`, beri tahu saya agar saya buatkan kodenya!



ini lanjutanya apakah masih perlu dibuat visualisasinya?

Berdasarkan gambar dokumen yang Anda lampirkan pada bagian **3.6.5 Interpretasi Hasil Korelasi**, jawabannya adalah **tidak memerlukan visualisasi grafik baru.** PNG

Bagian ini berfokus pada penjelasan naratif mengenai arah hubungan (positif/negatif), pencocokan nilai koefisien korelasi ke dalam **Tabel 3.4 (Kriteria kekuatan hubungan berdasarkan interval nilai)**, serta penarikan kesimpulan secara konseptual. PNG + 1

Visualisasi yang relevan untuk bagian korelasi sudah diwakili oleh *scatter plot* yang dibuat pada tahap analisis korelasi sebelumnya. Pada bagian ini, Anda cukup menyajikan hasil perhitungan angka koefisien korelasi ke dalam tabel interpretasi berdasarkan kriteria Sugiyono tersebut.
PNG

Apakah ada bagian lain atau kode program terkait interpretasi ini yang ingin saya bantu buatkan?

buatkan saya perhitungan secara statistikanya

Tampilkan kode

Berikut adalah hasil lengkap **perhitungan statistika** secara utuh (mencakup Statistika Deskriptif, Uji Normalitas Shapiro-Wilk, Analisis Korelasi, hingga Pengujian Hipotesis) untuk **250 pasien** pertama pada variabel `age`, `bp`, dan `sc` sesuai dengan seluruh persamaan yang tertera pada dokumen Anda:

1. Tabel Ringkasan Statistika Deskriptif

Perhitungan ini menggunakan persamaan Mean ($\bar{x}$) **Persamaan 3-1**, Varians Sampel (s^2) **Persamaan 3-2**, dan Standar Deviasi Sampel (s) **Persamaan 3-3**:

ariabel	Jumlah (n)	Mean ($\bar{x}$)	Median	Modus
sia (`age`)	250	54.56	59.00	60.00
ekanan Darah (`bp`)	250	79.64	80.00	70.00
reatinin Serum (`sc`)	250	4.27	2.20	1.30

maksud saya buatkan saya dalam hitungan di phyton

Berikut adalah kode Python lengkap yang menggabungkan seluruh perhitungan statistika di atas (Statistika Deskriptif, Uji Normalitas Shapiro-Wilk, Analisis Korelasi, Uji Hipotesis t, hingga Interpretasi Kekuatan Hubungan) untuk 250 pasien pertama pada variabel `age`, `bp`, dan `sc`:

Python

```python
import numpy as np
import pandas as pd
import scipy.stats as stats

# 1. Membaca file Excel dan mengambil 250 data pertama
df = pd.read_excel('3TIB_03_ChronicKidneyDisease.xlsx')
df_250 = df.head(250)

kolom_target = ['age', 'bp', 'sc']
labels = {
    'age': 'Usia (age)',
    'bp': 'Tekanan Darah (bp)',
    'sc': 'Kreatinin Serum (sc)',
}

print('=' * 80)
print('HASIL PERHITUNGAN STATISTIKA PYTHON (250 PASIEN)')
print('=' * 80)

# --- A. STATISTIKA DESKRIPTIF ---
print('\n[1] TABEL STATISTIKA DESKRIPTIF')
desc_list = []
for col in kolom_target:
    data = df_250[col].dropna()
```

```
# --- A. STATISTIKA DESKRIPTIF ---
print('\n[1] TABEL STATISTIKA DESKRIPTIF')
desc_list = []
for col in kolom_target:
  data = df_250[col].dropna()
  n = len(data)
  mean_val = data.mean()  # Persamaan 3-1
  median_val = data.median()
  mode_val = data.mode()[0] if not data.mode().empty else np.nan
  min_val = data.min()
  max_val = data.max()
  range_val = max_val - min_val
  q1 = data.quantile(0.25)
  q3 = data.quantile(0.75)
  var_val = data.var(ddof=1)  # Persamaan 3-2 (s^2)
  std_val = data.std(ddof=1)  # Persamaan 3-3 (s)

  desc_list.append({
      'Variabel': labels[col],
      'n': n,
      'Mean': round(mean_val, 2),
      'Median': round(median_val, 2),
      'Modus': round(mode_val, 2),
      'Min': min_val,
      'Max': max_val,
      'Range': range_val,
      'Q1': round(q1, 2),
      'Q3': round(q3, 2),
      'Varians (s^2)': round(var_val, 2),
      'Std Dev (s)': round(std_val, 2),
  })

df_desc = pd.DataFrame(desc_list)
print(df_desc.to_string(index=False))

# --- B. UJI NORMALITAS (SHAPIRO-WILK) ---
```

```
# --- B. UJI NORMALITAS (SHAPIRO-WILK) ---
print('\n[2] UJI NORMALITAS SHAPIRO-WILK (alpha = 0.05)')
norm_list = []
for col in kolom_target:
  data = df_250[col].dropna()
  w_stat, p_val = stats.shapiro(data)  # Persamaan 3-4 (W)
  status = (
      'Berdistribusi Normal' if p_val >= 0.05 else 'Tidak Berdistribusi Normal'
  )
  norm_list.append({
      'Variabel': labels[col],
      'Statistik W': round(w_stat, 4),
      'p-value': f'{p_val:.4e}',
      'Kesimpulan': status,
  })
df_norm = pd.DataFrame(norm_list)
print(df_norm.to_string(index=False))

# --- C. ANALISIS KORELASI & PENGUJIAN HIPOTESIS ---
print('\n[3] ANALISIS KORELASI & PENGUJIAN HIPOTESIS (Uji t)')
pairs = [('age', 'bp'), ('age', 'sc'), ('bp', 'sc')]
corr_list = []

for x, y in pairs:
  data = df_250[[x, y]].dropna()
  nx = len(data)

  # Cek normalitas untuk menentukan Pearson atau Spearman
  _, px = stats.shapiro(data[x])
  _, py = stats.shapiro(data[y])

  if px >= 0.05 and py >= 0.05:
    r, p_val = stats.pearsonr(data[x], data[y])  # Persamaan 3-5 (r)
    metode = 'Pearson (r)'
  else:
    r, p_val = stats.spearmanr(data[x], data[y])  # Persamaan 3-6 (rho)
    metode = 'Spearman (rho)'

  # Statistik Uji t (Persamaan 3-7)
```

```python
    # Statistik uji t (rumus 3.7)
    if abs(r) == 1.0:
        t_stat = np.inf
    else:
        t_stat = (r * np.sqrt(nx - 2)) / np.sqrt(1 - r**2)

    hipotesis = (
        'H0 Ditolak (Ada hubungan yang signifikan)'
        if p_val < 0.05
        else 'H0 Diterima (Tidak ada hubungan yang signifikan)'
    )

    # Kriteria Kekuatan Sugiyono (Tabel 3.4)
    abs_r = abs(r)
    if abs_r <= 0.199:
        kekuatan = 'Sangat rendah'
    elif abs_r <= 0.399:
        kekuatan = 'Rendah'
    elif abs_r <= 0.599:
        kekuatan = 'Sedang'
    elif abs_r <= 0.799:
        kekuatan = 'Kuat'
    else:
        kekuatan = 'Sangat kuat'

    corr_list.append({
        'Pasangan': f'{x} - {y}',
        'Metode': metode,
        'Koefisien': round(r, 4),
        't-hitung': round(t_stat, 4),
        'p-value': f'{p_val:.4e}',
        'Kekuatan': kekuatan,
        'Keputusan Uji': hipotesis,
    })

df_corr = pd.DataFrame(corr_list)
print(df_corr.to_string(index=False))
print('=' * 80)
```